

NEHA SHARMA U24AI050 LAB 12

10 April 2026 16:24

Objective

To implement and compare different memory allocation algorithms:

1. First Fit
2. Next Fit
3. Best Fit
4. Worst Fit
5. Quick Fit

Problem Statement

Simulate dynamic memory allocation in an operating system using different strategies. Analyze their performance in terms of:

- Memory utilization
- Fragmentation (internal & external)
- Allocation speed

Tasks

1. Implement the Following Algorithms

Write a C program that implements:

1. First Fit – Allocates the first available block that is large enough.
2. Next Fit – Similar to First Fit but starts searching from the last allocated block.
3. Best Fit – Allocates the smallest block that is large enough.
4. Worst Fit – Allocates the largest available block.
5. Quick Fit – Maintains separate free lists for common block sizes.

2. Simulation & Analysis

- Generate random process sizes (e.g., 16KB–256KB).
- Simulate allocation and deallocation.
- Track:
 - o Success rate (percentage of successful allocations)
 - o Memory utilization (used vs. free memory)
 - o Fragmentation (wasted memory due to small gaps)
 - o Execution time (speed of allocation)

3. Comparison & Report

- Compare the algorithms based on the collected metrics.
- Discuss which strategy performs best under different scenarios.

Lab Requirements

1. Program Structure

- Use a linked list to represent memory blocks.
- Implement functions for:
 - o initialize_memory()
 - o allocate_memory(strategy, size)
 - o free_memory(process_id)
 - o display_memory()
 - o coalesce_free_blocks() (to merge adjacent free blocks)

2. Expected Output

- Memory status after each allocation/deallocation.
- Statistics (success rate, fragmentation, time taken).

3. Sample Test Cases

- Test Case 1: Allocate 5 processes (e.g., 64KB, 128KB, 32KB, 256KB, 16KB).
- Test Case 2: Free some processes and reallocate.
- Test Case 3: Stress test with 20+ random allocations.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
#define MAX 50
```

```
typedef struct Block {  
    int size;  
    int free;  
    struct Block* next;  
} Block;
```

```
// Create fresh memory for each algorithm
```

```
Block* create_memory(int total) {  
    Block* head = (Block*)malloc(sizeof(Block));  
    head->size = total;  
    head->free = 1;  
    head->next = NULL;  
    return head;  
}
```

```
// Split block
```

```

void split(Block* fit, int size) {
    if (fit->size > size) {
        Block* newBlock = (Block*)malloc(sizeof(Block));
        newBlock->size = fit->size - size;
        newBlock->free = 1;
        newBlock->next = fit->next;

        fit->size = size;
        fit->next = newBlock;
    }
    fit->free = 0;
}

```

// Coalesce

```

void coalesce(Block* head) {
    Block* temp = head;
    while (temp && temp->next) {
        if (temp->free && temp->next->free) {
            temp->size += temp->next->size;
            temp->next = temp->next->next;
        } else {
            temp = temp->next;
        }
    }
}

```

// ----- ALGORITHMS -----

// First Fit

```

int first_fit(Block* head, int size) {
    Block* temp = head;
    while (temp) {
        if (temp->free && temp->size >= size) {
            split(temp, size);
            return 1;
        }
        temp = temp->next;
    }
    return 0;
}

```

```

// Next Fit
int next_fit(Block* head, int size) {
    static Block* last = NULL;
    if (!last) last = head;

    Block* temp = last;

    do {
        if (temp->free && temp->size >= size) {
            split(temp, size);
            last = temp;
            return 1;
        }
        temp = temp->next ? temp->next : head;
    } while (temp != last);

    return 0;
}

```

```

// Best Fit
int best_fit(Block* head, int size) {
    Block* temp = head;
    Block* best = NULL;

    while (temp) {
        if (temp->free && temp->size >= size) {
            if (!best || temp->size < best->size)
                best = temp;
        }
        temp = temp->next;
    }

    if (best) {
        split(best, size);
        return 1;
    }
    return 0;
}

```

```

// Worst Fit
int worst_fit(Block* head, int size) {
    Block* temp = head;
    Block* worst = NULL;

    while (temp) {
        if (temp->free && temp->size >= size) {
            if (!worst || temp->size > worst->size)
                worst = temp;
        }
        temp = temp->next;
    }

    if (worst) {
        split(worst, size);
        return 1;
    }
    return 0;
}

// ----- SIMULATION -----

void simulate(char* name, int (*alloc)(Block*, int), int processes[]) {
    Block* mem = create_memory(1024);

    int success = 0;
    int used = 0, free_mem = 0, fragments = 0;

    clock_t start = clock();

    for (int i = 0; i < MAX; i++) {
        if (alloc(mem, processes[i]))
            success++;

        if (i % 5 == 0 && i > 0) {
            coalesce(mem);
        }
    }

    clock_t end = clock();

```

```

// stats
Block* temp = mem;
while (temp) {
    if (temp->free) {
        free_mem += temp->size;
        if (temp->size < 32)
            fragments++;
    } else {
        used += temp->size;
    }
    temp = temp->next;
}

double time_taken = (double)(end - start) / CLOCKS_PER_SEC;

printf("\n==== %s ====\n", name);
printf("Success Rate: %.2f%%\n", success * 100.0 / MAX);
printf("Used: %d KB | Free: %d KB\n", used, free_mem);
printf("Fragments: %d\n", fragments);
printf("Time: %f sec\n", time_taken);
}

// ----- MAIN -----

int main() {
    srand(time(NULL));

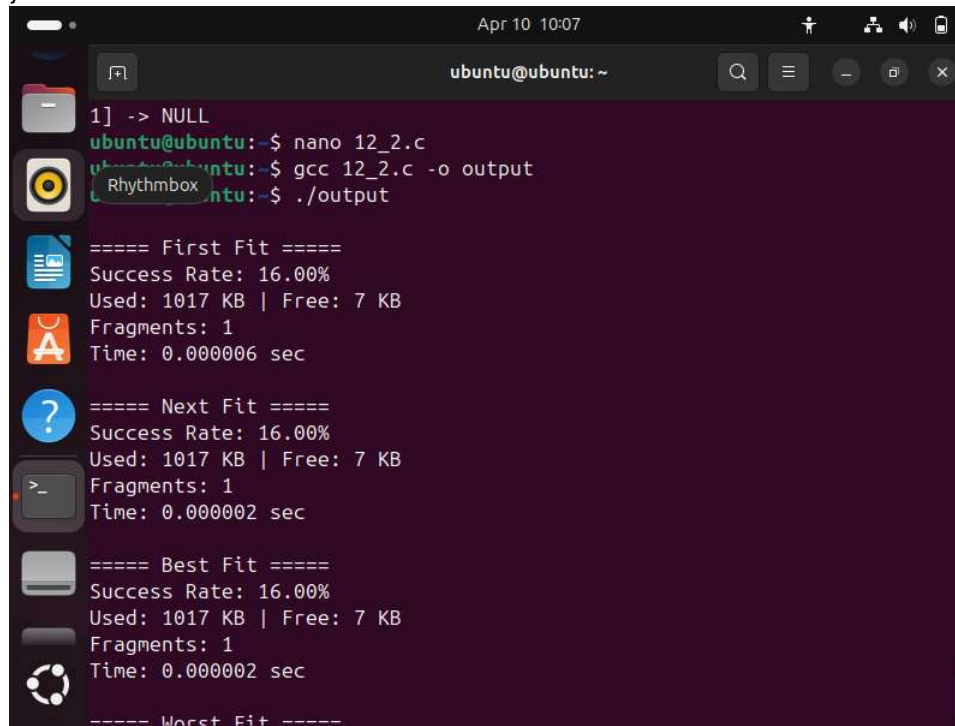
    int processes[MAX];

    // random processes (16–256 KB)
    for (int i = 0; i < MAX; i++) {
        processes[i] = (rand() % 241) + 16;
    }

    simulate("First Fit", first_fit, processes);
    simulate("Next Fit", next_fit, processes);
    simulate("Best Fit", best_fit, processes);
    simulate("Worst Fit", worst_fit, processes);
}

```

```
return 0;  
}
```



A terminal window titled 'ubuntu@ubuntu: ~' with a search icon, menu icon, and window control buttons. The terminal shows the following commands and output:

```
1] -> NULL  
ubuntu@ubuntu:~$ nano 12_2.c  
ubuntu@ubuntu:~$ gcc 12_2.c -o output  
ubuntu@ubuntu:~$ ./output
```

The output of the program is as follows:

```
==== First Fit ====  
Success Rate: 16.00%  
Used: 1017 KB | Free: 7 KB  
Fragments: 1  
Time: 0.000006 sec  
  
==== Next Fit ====  
Success Rate: 16.00%  
Used: 1017 KB | Free: 7 KB  
Fragments: 1  
Time: 0.000002 sec  
  
==== Best Fit ====  
Success Rate: 16.00%  
Used: 1017 KB | Free: 7 KB  
Fragments: 1  
Time: 0.000002 sec  
  
==== Worst Fit ====
```

```
Apr 10 10:08
ubuntu@ubuntu: ~
===== Next Fit =====
Success Rate: 16.00%
Used: 1017 KB | Free: 7 KB
Fragments: 1
Time: 0.000002 sec
===== Best Fit =====
Success Rate: 16.00%
Used: 1017 KB | Free: 7 KB
Fragments: 1
Time: 0.000002 sec
===== Worst Fit =====
Success Rate: 16.00%
Used: 1017 KB | Free: 7 KB
Fragments: 1
Time: 0.000002 sec
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$
ubuntu@ubuntu:~$
```